

TPTL-DIST: A Calculus for Verifying Real-Time Distributed Systems

Javier Enríquez Mendoza¹, Sam Speight², and Vincent Rahli³

¹ University of Birmingham `jxe275@student.bham.ac.uk`

² University of Birmingham `s.l.speight@bham.ac.uk`

³ University of Birmingham `V.Rahli@bham.ac.uk`

Abstract. We introduce TPTL-DIST, a temporal logic and labelled sequent calculus for specifying and verifying real-time distributed systems, formalised in Agda to provide machine-checked correctness and tool support. The logic extends Timed Propositional Temporal Logic (TPTL) with past operators, freeze quantification, comparisons between time expressions, domain-specific atoms, and an induction principle on time. We provide a world-based semantics, develop an interval-annotated calculus, and establish its soundness w.r.t. the semantics. To demonstrate its expressiveness, we encode standard distributed system patterns, and we carry out a mechanized proof of a key real-time property of the Pistis Byzantine broadcast protocol. Our results show that TPTL-DIST provides a uniform, mechanized framework for reasoning about temporal properties of distributed systems.

Keywords: Temporal Logic, Real-Time, Distributed Systems, Byzantine Reliable Broadcast, Agda

1 Introduction

Modern systems, such as smart factories, power grids, high-speed payment processing (e.g., Visa, PayPal), ultra-low-latency 5G networks, and mission-critical air traffic control, rely on distributed systems. In these settings, correctness depends not only on what actions occur, but when they occur as delays can lead to failures. While temporal logics such as LTL and CTL provide well established foundations for reasoning about system behaviour over time, they lack support for real-time properties. Extensions such as TPTL [1] and Timed CTL [2] incorporate clocks and timing constraints, and tools like PRISM [3] and TLA++ [4–6] support modelling and model checking of real-time and distributed systems [7–15]. However, existing formalisms differ from our approach in important ways and face key limitations. (1) Our work leverages a full Agda formalisation, enabling machine-checked reasoning and providing a foundation for verified tool support. (2) Logics like TPTL and Timed CTL cannot express common distributed system events such as message send/receive or agent correctness, and it lacks inductive reasoning over time. (3) Tools like PRISM and TLA+ require

exhaustively exploring all possible states of the system, which is not always feasible. (4) Existing approaches separate clock reasoning from the communication structure of distributed systems.

To complement the current landscape of approaches, we introduce TPTL-DIST, a fully formalised temporal logic and labelled sequent calculus designed specifically for reasoning about real-time fault-tolerant distributed protocols. The logic extends TPTL with past operators, freeze quantification, a general comparison operator on time expressions, and atomic predicates encoding message transmission, internal events, and agent correctness.

TPTL-DIST’s rules are annotated with time intervals and the semantics is given over an abstract world structure equipped with an induction principle for temporal reasoning. A central goal of TPTL-DIST is to provide a uniform framework to state and reason about both temporal and distributed aspects of a protocol. All definitions, rules, and meta-theoretical results are formalized in Agda: <https://github.com/jaeem006/TPTL-DIST/blob/main/Index2025.lagda>. We use the symbol  to link to the formalization.

Contributions Our contributions are as follows, demonstrating that TPTL-DIST provides a principled and mechanized foundation for reasoning about real-time distributed systems:

- TPTL-DIST a new logic for real-time distributed systems, extending TPTL with past operators, quantifiers, comparisons on time expressions, and fault-tolerant distributed system operators.
- A sequent calculus for TPTL-DIST that supports reasoning with intervals, which is sound for the intended semantics.
- A general semantic framework based on ordered worlds, enabling a principled induction rule for unbounded temporal reasoning.
- A mechanization in Agda, covering the logic, calculus and meta-theory.
- A modular design of the Pistis real-time reliable broadcast protocol [16].
- A mechanized proof of Pistis’ timeliness property, showcasing TPTL-DIST’s support for real-time constraints, message-passing patterns, and induction.

2 Related Work

Several proof systems have been proposed for LTL-like logics. However, the until operator is known to pose difficulties due to its dual character of being both universal and existential. A labeled natural deduction system was proposed in [17] to treat those difficulties. Simplifying and generalizing that system (see Sec. 3), we propose a labeled sequent calculus that allows reasoning about TPTL’s temporal operators, in particular until and freeze, in a uniform way.

TPTL The syntax of TPTL is as follows, where $\mathbf{U}$ and $\bigcirc$ are the usual “until” and “next” LTL operators; p is an atomic proposition; x is a clock variable; c is

a clock expression; $\sim \in \{\leq, <, =, >, \geq\}$ allows comparing a clock variable with a clock expression; and the *freeze* formula $x \cdot \varphi$ binds the current time to x in φ :

$$\varphi ::= p \mid \varphi \wedge \varphi \mid \neg \varphi \mid \varphi \mathbf{U} \varphi \mid \bigcirc \varphi \mid x \sim c \mid x \cdot \varphi$$

The *eventually* and *always* operators are defined as usual: $\Diamond \varphi \equiv \top \mathbf{U} \varphi$ and $\Box \varphi \equiv \neg \Diamond \neg \varphi$, while the freeze operator allows capturing that φ eventually holds by time c : $x \cdot \Diamond(x \leq c \wedge \varphi)$. To understand this, consider TPTL's semantics over the discrete notion of time given by $\mathbb{N}$. A formula's semantics is given in terms of a mapping ρ from $\mathbb{N}$ to states, a point in time $i \in \mathbb{N}$, and a variable valuation v , where a state ρ_n is a pair of a valuation on atoms σ_n and a timestamp τ_n :

$$\begin{aligned} \rho, i, v \models p &\iff p \in \sigma_i & \rho, i, v \models \varphi \wedge \psi &\iff \rho, i, v \models \varphi \text{ and } \rho, i, v \models \psi \\ \rho, i, v \models \neg \varphi &\iff \rho, i, v \not\models \varphi & \rho, i, v \models \bigcirc \varphi &\iff \rho, i+1, v \models \varphi \\ \rho, i, v \models x \sim c &\iff \tau_i - v(x) \sim c & \rho, i, v \models x \cdot \varphi &\iff \rho, i, v[x \mapsto \tau_i] \models \varphi \\ \rho, i, v \models \varphi \mathbf{U} \psi &\iff \exists j > i. (\rho, j, v \models \psi) \wedge \forall k \in [i, j). (\rho, k, v \models \varphi) \end{aligned}$$

The semantics of “ φ eventually holds by time c ”, i.e., $x \cdot \Diamond(x \leq c \wedge \varphi)$ w.r.t. ρ, i, v states that there exists a time $\tau_j \leq \tau_i + c$ at which φ is true, for some $j > i$. This is achieved by “freezing” the current time τ_i , by mapping x to τ_i in the variable valuation v ; by jumping to a later time τ_j thanks to the $\Diamond$ operator; and finally the constraint $x \leq c$ guarantees that $\tau_j \leq \tau_i + c$.

Despite its expressive power, TPTL does not allow formalizing properties of distributed systems such as those relevant to the Pistis real-time Byzantine-resilient reliable broadcast protocol [16], lacking the support for reasoning about agents, messages, communication, etc. TPTL-DIST is designed to address these points, as well as TPTL's lack of support for important time-related reasoning mechanisms, such as reasoning about past properties in addition to future properties, and reasoning inductively over time.

Note that TPTL-DIST aims to support the specification and verification of systems that are required to perform critical actions in a timely fashion. As such, it allows modeling a *real* or *physical*, quantitative notion of time, in addition to a *logical*, qualitative notion of time [18, 19].

Other approaches with similar motivation, i.e., formally verifying timing properties of systems, but using different techniques have been proposed in the literature. We illustrate below how our approach relates and differs.

TLA+: *Temporal Logic of Actions* The primary distinction with TLA+ lies in its treatment of time. Our system integrates temporal constructs directly into the language's syntax through its freeze and comparison operators. This representation of time allows for simple and intuitive modeling of temporal conditions. By contrast, TLA+ relies on temporal operators such as *always* and *eventually* to express time-dependent properties. Although it is feasible to capture a quantitative notion of time in TLA+ using variables that represent time, these variables are treated as ordinary state variables and are not inherently connected to the temporal operators. As a result, while providing great flexibility, the burden of correctly managing time relies on the user explicitly updating those variables, as opposed to the automated management of time in TPTL-DIST.

PRISM: Probabilistic Model Checking PRISM is primarily designed for the verification of probabilistic properties. It is based on probabilistic temporal logics such as PCTL (Probabilistic Computation Tree Logic) [20, 21] and CSL (Continuous Stochastic Logic) [22, 23]. In addition to including temporal mechanisms that enable reasoning about time-related properties, PRISM’s core strength lies in its ability to handle systems characterized by randomness, uncertainty, and non-determinism. TPTL-DIST complements PRISM’s approach, whose model checking engine imposes some restrictions on the real-time systems and properties that can be verified, providing flexibility to the users thanks to its calculus.

Summary TLA+ lets users manage time flexibly through variables, requiring careful handling, while PRISM takes a more restricted approach to enable effective model checking. TPTL-DIST provides primitives that do not require careful management of variables, while still providing some flexibility when stating time-related properties, along with a calculus to prove those properties.

3 TPTL-DIST

We now present TPTL-DIST, an extension of TPTL + Past [24], with quantifiers, a more general comparison operator, a “discrete” semantics, and a sequent calculus.

3.1 Syntax

To reason about distributed systems, which are composed of nodes, or agents, communicating by exchanging messages, we fix some types (T) to allow quantifying over agents (**AGENT**), sets of agents (**AGENTS**), and data (**DATA**), where **DATA** is used to capture the information exchanged between agents $\clubsuit$:

$$T \in \text{Type} ::= \mathbf{AGENT} \mid \mathbf{AGENTS} \mid \mathbf{DATA}$$

Let v_a range over a set $\mathcal{V}_a$ of variables of type **AGENT** and c_a range over a set $\mathcal{C}_a$ of constants of type **AGENT**; let v_A range over a set $\mathcal{V}_A$ of variables of type **AGENTS**; and let v_d range over a set $\mathcal{V}_d$ of variables of type **DATA** and c_d range over a set $\mathcal{C}_d$ of constants of type **DATA**. Expressions of types **AGENT** $\clubsuit$, **AGENTS** $\clubsuit$, and **DATA** $\clubsuit$ are defined as follows:

$$i, j \in \mathbf{Agent} ::= v_a \mid c_a \quad A \in \mathbf{Agents} ::= v_A \mid [i_1, \dots, i_n] \quad d \in \mathbf{Data} ::= v_d \mid c_d$$

TPTL-DIST also includes expressions to capture time constraints, where v_t ranges over a set $\mathcal{V}_t$ of time variables, **0** is a constant, capturing an empty amount of time, and $\bullet$ is a binary operation, intuitively adding two time expressions $\clubsuit$:

$$t \in \text{TimeExp} ::= v_t \mid \mathbf{0} \mid t \bullet t$$

To compare time expressions, we further introduce the following operators $\clubsuit$:

$$\sim \in \text{Comparison} ::= \sqsubset \mid \equiv \mid \triangleleft$$

Intuitively, $t_1 \sqsubset t_2$ states that t_1 is strictly lower than t_2 , $t_1 \equiv t_2$ states that t_1 and t_2 are equal, and $t_1 \triangleleft t_2$ states that t_1 is t_2 's immediate predecessor.

To reason about distributed systems, one must reason about the knowledge acquired by agents over time. The two main approaches are the *implicit* or *external* representation of knowledge, which is externally assigned to agents; and the *explicit* representation, where knowledge is explicitly “stored” in states, allowing agents to compute and act on their knowledge [25]. We take an explicit approach, introducing a number of atomic expressions that capture knowledge through the exchange of messages between agents. A *distributed* atom a is either a *send* atom $\text{send}(i, d, A)$, stating that agent i sent the data d to the set of agents A ; or a *receive* atom $\text{recv}(i, d, j)$, stating that agent i received the data d from agent j ; or an *internal event* atom $\text{internal}(i, d)$, stating that d happened at agent i ; or a *correct* atom $\text{correct}(i)$, stating that agent i is correct $\star$:

$$a_d \in \text{DistAtom} ::= \text{send}(i, d, A) \mid \text{recv}(i, d, j) \mid \text{internal}(i, d) \mid \text{correct}(i)$$

Furthermore, TPTL-DIST includes atoms to reason about sets of agents as follows, where $n \in \mathbb{N}$, $\text{size}(A, n)$ states that the collection of agents A is of size n , and $i \in A$ states that agent i is in the collection of agents A $\star$:

$$a_s \in \text{SetAtom} ::= \text{size}(A, n) \mid i \in A$$

An atom is then either a *distributed* or a *set* atom: $a \in \text{Atom} ::= a_d \mid a_s$.

TPTL-DIST's syntax includes: (1) propositional connectives; (2) past and future LTL operators; (3) a freeze operator along with a comparison operator on time expressions; (4) quantifiers; (5) atoms to reason about distributed systems. It is defined as follows, where $\mathbf{S}$ and $\mathbf{Y}$ are the “since” and “yesterday” past counterparts of $\mathbf{U}$ and $\mathbf{O}$, and v is a variable of type T in $\forall v : T. \varphi$ and $\exists v : T. \varphi$ $\star$:

$$\begin{aligned} \varphi \in \text{Formula} ::= & a \mid \top \mid \perp \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \varphi \rightarrow \varphi \mid \varphi \mathbf{U} \varphi \mid \mathbf{O} \varphi \\ & \mid \varphi \mathbf{S} \varphi \mid \mathbf{Y} \varphi \mid t \sim t \mid v_t \cdot \varphi \mid \forall v : T. \varphi \mid \exists v : T. \varphi \end{aligned}$$

We write $\forall v_1, \dots, v_n : T. \varphi$ for $\forall v_1 : T. \dots \forall v_n : T. \varphi$, and similarly for $\exists$. Negation is defined as usual: $\neg \varphi \equiv \varphi \rightarrow \perp$. We recall the definitions of *eventually* and *always* (in white) as well as their past counterparts (in black) $\star$:

$$\Diamond \varphi \equiv \top \mathbf{U} \varphi \quad \Box \varphi \equiv \neg \Diamond \neg \varphi \quad \blacklozenge \varphi \equiv \top \mathbf{S} \varphi \quad \blacksquare \varphi \equiv \neg \blacklozenge \neg \varphi$$

Thanks to TPTL-DIST's general comparison operator $t_1 \sim t_2$, a time-bounded version of $\Diamond$ can be defined more straightforwardly than in TPTL: $\Diamond_t \varphi \equiv v_t \cdot \Diamond(v'_t \cdot v'_t \sqsubseteq (v_t \bullet t) \wedge \varphi)$, stating that φ happens sometime in the future by time t , where $t_1 \sqsubseteq t_2$ stands for $t_1 \sqsubset t_2 \vee t_1 \equiv t_2$. Similarly, we define a time-bounded version of the $\Box$ operator: $\Box_t \varphi \equiv v_t \cdot \Box(v'_t \cdot v'_t \sqsubseteq (v_t \bullet t) \rightarrow \varphi)$ stating that φ always holds between now and t . Furthermore, we define the following *strict* version of the *sometimes before* operator $\blacklozenge$ as follows: $\blacktriangleleft \varphi \equiv \mathbf{Y}(\blacklozenge \varphi)$, stating that φ happened sometimes in the past, but not now $\star$.

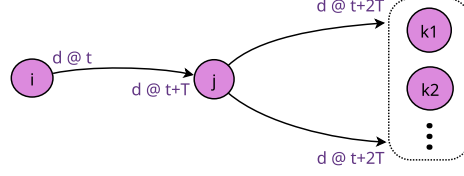


Fig. 1. Forwarding example

3.2 Forwarding Example

We now illustrate aspects of the language using a simple forwarding example, demonstrating how to capture message passing patterns in distributed systems, where agents exchange information by sending/receiving messages (see Appx. A for further examples). In this example, messages are relayed between agents of a synchronous system. Sec. 3.6 then proves a property of this system in detail, illustrating the use of TPTL-DIST's rules presented in Sec. 3.5.

As illustrated in Fig. 1, in this example, an agent *i* sends a message *d* at time *t* to another agent *j*, which then forwards *d* to all other agents. We are then guaranteed that any agent would have received that message by time *t* + 2*T* assuming that the maximum transmission delay between agents is *T*.

All agents are assumed to be correct and communication is assumed to be synchronous, ensuring that all sent messages are received within a time interval *T*, captured by the following formula stating that if an agent sends a message *d* at time *t* to a collection of agents, those agents will receive *d* by time *t* + *T* ⚙:

$$\forall i, j : \mathbf{AGENT}. \forall d : \mathbf{DATA}. \forall A : \mathbf{AGENTS}. \quad (1) \\ \square(\text{send}(i, d, A) \rightarrow \Diamond_T(j \in A \rightarrow \text{recv}(j, d, i)))$$

Additionally, we assume that agents immediately forward the information they receive to all other agents, captured by the following formula stating that whenever *j* receives a message it immediately sends it to *k*, and this for any *k* ⚙:

$$\forall i, j, k : \mathbf{AGENT}. \forall d : \mathbf{DATA}. \square(\text{recv}(j, d, i) \rightarrow \text{send}(j, d, [k])) \quad (2)$$

As shown in Sec. 3.6, using TPTL-DIST's calculus presented below, one can then derive the following formula from the above assumptions, stating that if an agent *i* sends a message *d* to some agent *j*, then any agent *k* will also receive *d* before 2*T* (where 2*T* is simply *T* • *T*) ⚙:

$$\forall i, j, k : \mathbf{AGENT}. \forall d : \mathbf{DATA}. \square(\text{send}(i, d, [j]) \rightarrow \Diamond_{2T} \text{recv}(k, d, j)) \quad (3)$$

3.3 Semantics

We now adapt and extend TPTL's semantics to our calculus, in particular to account for domain-specific atoms and a more general comparison operator between time expressions. Formulas are interpreted w.r.t. a collection of worlds,

$\cdot$ -sym	$\forall w_1, w_2 : \mathcal{W}. w_1 \cdot w_2 = w_2 \cdot w_1$
$\cdot$ -assoc	$\forall w_1, w_2, w_3 : \mathcal{W}. w_1 \cdot (w_2 \cdot w_3) = (w_1 \cdot w_2) \cdot w_3$
$\preceq$ -trans	$\forall w_1, w_2, w_3 : \mathcal{W}. w_1 \preceq w_2 \rightarrow w_2 \preceq w_3 \rightarrow w_1 \preceq w_3$
$\cdot$ -unit	$\forall w : \mathcal{W}. 0 \cdot w = w$
$\preceq$ -refl	$\forall w : \mathcal{W}. w \preceq w$
0-initial	$\forall w : \mathcal{W}. 0 \preceq w$
monotonicity	$\forall w_1, w_2, w_3, w_4 : \mathcal{W}. w_1 \preceq w_2 \rightarrow w_3 \preceq w_4 \rightarrow w_1 \cdot w_3 \preceq w_2 \cdot w_4$
sub	$\forall w_1, w_2 : \mathcal{W}. w_1 \triangleleft w_2 \rightarrow w_1 \preceq w_2$
right-density	$\forall w_1, w_2 : \mathcal{W}. w_1 \preceq w_2 \rightarrow w_1 = w_2 \vee \exists w : \mathcal{W}. w_1 \preceq w \wedge w \triangleleft w_2$
left-density	$\forall w_1, w_2 : \mathcal{W}. w_1 \preceq w_2 \rightarrow w_1 = w_2 \vee \exists w : \mathcal{W}. w_1 \triangleleft w \wedge w \preceq w_2$
conditional-totality	$\forall w_1, w_2, w : \mathcal{W}. w_1 \preceq w \rightarrow w_2 \preceq w \rightarrow w_1 \preceq w_2 \vee w_2 \preceq w_1$
induction	$\forall P : \mathcal{W} \rightarrow \text{Set}.$ $(\forall w : \mathcal{W}. (\forall u, v : \mathcal{W}. u \preceq v \rightarrow v \triangleleft w \rightarrow P(u)) \rightarrow P(w)) \rightarrow$ $\forall w : \mathcal{W}. P(w)$

Fig. 2. World axioms

which provides meaning to time expressions. The set of worlds $\mathcal{W}$ includes a constant 0 and a binary operation $\cdot$ (the semantic counterparts of the time expressions in **TimeExp**) and is equipped with two relations $\preceq$ and $\triangleleft$ (the semantic counterparts of the relations in **Comparison**) satisfying the axioms in Fig. 2 .

The axioms $\cdot$ -sym, $\cdot$ -assoc, $\cdot$ -unit, $\preceq$ -refl, $\preceq$ -trans, *monotonicity* and *0-initial* make $\mathcal{W}$ into a positively ordered (commutative) monoid [26] (though note that we do not require antisymmetry). Ordered commutative monoids have been exhibited as the mathematical structures of resource combination and convertibility [27]. Later, we will interpret time expressions as worlds, and, as such, may think of time as a resource.

We think of $\triangleleft$ as capturing immediate predecessor in the $\preceq$ relation; hence the *sub* axiom simply expresses that the former is a subrelation of the latter. Axiom *right-density* is used, in particular, along with the induction axiom, to derive the induction rule presented in Sec. 3.5, which is key to proving our main application in Lem. 2. Axiom *conditional-totality*, in the presence of $\preceq$ -refl and *monotonicity*, is equivalent to totality, by setting $w = w_1 \cdot w_2$.

The *right-density* and *left-density* axioms are in particular used to prove the left-to-right and right-to-left implications of $(\blacklozenge(\mathbf{Y} \varphi)) \leftrightarrow (\mathbf{Y}(\blacklozenge \varphi))$, respectively. Interestingly, only the former was necessary in Sec. 4.2.

Following [25, 28–30], formulas are interpreted w.r.t. a model, which is a tuple of the form $\langle r, \pi, w, \mathcal{V} \rangle$ consisting of (1) a run r , (2) an interpretation function π , which assigns meaning to atomic formulas, (3) a world $w \in \mathcal{W}$, and (4) a variable valuation $\mathcal{V}$ assigning meaning to free variables. A run r is a function in $\mathcal{W} \rightarrow \text{Global}$, where **Global** is a global state, ie., an element of $\text{List}(\text{ClosedAtom})$, where **ClosedAtom** is the type of closed atoms (closed elements of **Atom**). An interpretation is a function in $\text{Global} \rightarrow \text{ClosedAtom} \rightarrow \text{Set}$, which checks whether a given atom is true in a given global state. A variable valuation is a function in $(\mathcal{V}_a \rightarrow \mathcal{C}_a) \cup (\mathcal{V}_A \rightarrow \text{List}(\mathcal{C}_a)) \cup (\mathcal{V}_d \rightarrow \mathcal{C}_d) \cup (\mathcal{V}_t \rightarrow \mathcal{W})$. We write

$\llbracket _ \rrbracket : \text{Type} \rightarrow \text{Set}$	$\llbracket _ \rrbracket_{\mathcal{V}} : \text{TimeExp} \rightarrow \mathcal{W}$	$\llbracket _ \rrbracket : \text{Comparison} \rightarrow \mathcal{W} \rightarrow \mathcal{W} \rightarrow \text{Set}$
$\llbracket \text{AGENT} \rrbracket \equiv \mathcal{C}_a$	$\llbracket v_t \rrbracket_{\mathcal{V}} \equiv \mathcal{V}(v_t)$	$\llbracket \sqsubset \rrbracket \equiv \prec$
$\llbracket \text{AGENTS} \rrbracket \equiv \text{List}(\mathcal{C}_a)$	$\llbracket 0 \rrbracket_{\mathcal{V}} \equiv 0$	$\llbracket \equiv \rrbracket \equiv =$
$\llbracket \text{DATA} \rrbracket \equiv \mathcal{C}_d$	$\llbracket t_1 \bullet t_2 \rrbracket_{\mathcal{V}} \equiv \llbracket t_1 \rrbracket_{\mathcal{V}} \cdot \llbracket t_2 \rrbracket_{\mathcal{V}}$	$\llbracket \triangleleft \rrbracket \equiv \triangleleft$
$\llbracket _ \rrbracket_{\mathcal{V}} : \text{Agents} \rightarrow \text{List}(\mathcal{C}_a)$	$\llbracket _ \rrbracket_{\mathcal{V}} : \text{Agent} \rightarrow \mathcal{C}_a$	$\llbracket _ \rrbracket_{\mathcal{V}} : \text{Data} \rightarrow \mathcal{C}_d$
$\llbracket v_A \rrbracket_{\mathcal{V}} \equiv \mathcal{V}(v_A)$	$\llbracket v_a \rrbracket_{\mathcal{V}} \equiv \mathcal{V}(v_a)$	$\llbracket v_d \rrbracket_{\mathcal{V}} \equiv \mathcal{V}(v_d)$
$\llbracket [i_1, \dots, i_n] \rrbracket_{\mathcal{V}} \equiv [\llbracket i_1 \rrbracket_{\mathcal{V}}, \dots, \llbracket i_n \rrbracket_{\mathcal{V}}]$	$\llbracket c_a \rrbracket_{\mathcal{V}} \equiv c_a$	$\llbracket c_d \rrbracket_{\mathcal{V}} \equiv c_d$
$\llbracket _ \rrbracket_{\mathcal{V}} : \text{DistAtom} \rightarrow \text{ClosedAtom}$		
$\llbracket \text{send}(i, d, A) \rrbracket_{\mathcal{V}} \equiv \text{send}(\llbracket i \rrbracket_{\mathcal{V}}, \llbracket d \rrbracket_{\mathcal{V}}, \llbracket A \rrbracket_{\mathcal{V}})$		
$\llbracket \text{recv}(i, d, j) \rrbracket_{\mathcal{V}} \equiv \text{recv}(\llbracket i \rrbracket_{\mathcal{V}}, \llbracket d \rrbracket_{\mathcal{V}}, \llbracket j \rrbracket_{\mathcal{V}})$		
$\llbracket \text{internal}(i, d) \rrbracket_{\mathcal{V}} \equiv \text{internal}(\llbracket i \rrbracket_{\mathcal{V}}, \llbracket d \rrbracket_{\mathcal{V}})$		
$\llbracket \text{correct}(i) \rrbracket_{\mathcal{V}} \equiv \text{correct}(\llbracket i \rrbracket_{\mathcal{V}})$		

Fig. 3. Semantics of types, time expressions, comparison operators, agents, collections of agents, data, and distributed atoms

$\mathcal{V}(X_1, \dots, X_n)$ to indicate that $\mathcal{V}$ is a variable valuation with domain including the free variables of the syntactic expressions X_1 to X_n . We also write $w_1 \prec w_2$ for $\exists w : \mathcal{W}. w_1 \triangleleft w \wedge w \preceq w_2$.

Fig. 3 introduces the semantics of time expressions $\llbracket t \rrbracket_{\mathcal{V}} \in \mathcal{W}$, types $\llbracket T \rrbracket \in \text{Set}$, atoms $\llbracket a \rrbracket_{\mathcal{V}} \in \text{ClosedAtom}$, agents $\llbracket i \rrbracket_{\mathcal{V}} \in \mathcal{C}_a$, collections of agents $\llbracket A \rrbracket_{\mathcal{V}} \in \text{List}(\mathcal{C}_a)$, data $\llbracket d \rrbracket_{\mathcal{V}} \in \mathcal{C}_d$, and comparison operators $\llbracket \sim \rrbracket \in \mathcal{W} \rightarrow \mathcal{W} \rightarrow \text{Set}$, while Fig. 4 defines the semantics of formulas. In particular, the semantics of an atom a_d , checks that the interpretation $\llbracket a_d \rrbracket_{\mathcal{V}}$ is in the global state returned by r at world w .

3.4 Sequents and Rules

Some proof systems have been proposed for LTL-like logics [31, 32]. However, the until operator is known to pose difficulties [33]. A labeled natural deduction system was proposed in [17] to circumvent those difficulties, following a long line of work on labeled deduction systems [34–37]. Simplifying and generalizing that system, we propose a labeled sequent calculus that allows reasoning about until and timing properties using the same machinery.

A sequent is of the form $\Gamma \vdash \varphi^a$, where Γ is a list of hypotheses, φ is a formula, and a is a time annotation. Time annotations are as follows, where $\bullet$ indicates no annotation, meaning the formula is time-irrelevant $\clubsuit$:

$$a ::= \bullet \mid t \mid [t_1, t_2] \mid [t_1, t_2) \mid (t_1, t_2] \mid (t_1, t_2)$$

Such a sequent states that φ holds at time a in the context Γ . An hypothesis is of the form φ^a . We write φ for the annotated formula $\varphi^{\bullet}$. Similar to [17], TPTL-DIST relies on intervals to capture the meaning of $\mathbf{U}$. The formula $\varphi \mathbf{U} \psi$

$r, \pi, w, \mathcal{V} \models a_d$	$\iff \pi(r(w))(\llbracket a_d \rrbracket_{\mathcal{V}})$
$r, \pi, w, \mathcal{V} \models i \in A$	$\iff \llbracket i \rrbracket_{\mathcal{V}} \in \llbracket A \rrbracket_{\mathcal{V}}$
$r, \pi, w, \mathcal{V} \models \mathbf{size}(A, n)$	$\iff \llbracket A \rrbracket_{\mathcal{V}} = n$
$r, \pi, w, \mathcal{V} \models \top$	$\iff \text{True}$
$r, \pi, w, \mathcal{V} \models \perp$	$\iff \text{False}$
$r, \pi, w, \mathcal{V} \models \neg \varphi$	$\iff \neg(r, \pi, w, \mathcal{V} \models \varphi)$
$r, \pi, w, \mathcal{V} \models \varphi \wedge \psi$	$\iff r, \pi, w, \mathcal{V} \models \varphi \text{ and } r, \pi, w, \mathcal{V} \models \psi$
$r, \pi, w, \mathcal{V} \models \varphi \vee \psi$	$\iff r, \pi, w, \mathcal{V} \models \varphi \text{ or } r, \pi, w, \mathcal{V} \models \psi$
$r, \pi, w, \mathcal{V} \models \varphi \rightarrow \psi$	$\iff r, \pi, w, \mathcal{V} \models \varphi \text{ implies } r, \pi, w, \mathcal{V} \models \psi$
$r, \pi, w, \mathcal{V} \models \forall v : T. \varphi$	$\iff r, \pi, w, \mathcal{V}[v \mapsto z] \models \varphi \text{ for all } z \in \llbracket T \rrbracket$
$r, \pi, w, \mathcal{V} \models \exists v : T. \varphi$	$\iff r, \pi, w, \mathcal{V}[v \mapsto z] \models \varphi \text{ for some } z \in \llbracket T \rrbracket$
$r, \pi, w, \mathcal{V} \models t_1 \sim t_2$	$\iff \llbracket t_1 \rrbracket_{\mathcal{V}} \llbracket \sim \rrbracket \llbracket t_2 \rrbracket_{\mathcal{V}}$
$r, \pi, w, \mathcal{V} \models v_t \cdot \varphi$	$\iff r, \pi, w, \mathcal{V}[v_t \mapsto w] \models \varphi$
$r, \pi, w, \mathcal{V} \models \varphi \mathbf{U} \psi$	$\iff \exists w' \succ w. (r, \pi, w', \mathcal{V} \models \psi) \wedge$ $\quad \forall w''. w \preceq w'' \prec w' \rightarrow (r, \pi, w'', \mathcal{V} \models \varphi)$
$r, \pi, w, \mathcal{V} \models \bigcirc \varphi$	$\iff \exists w' \triangleright w. r, \pi, w', \mathcal{V} \models \varphi$
$r, \pi, w, \mathcal{V} \models \varphi \mathbf{S} \psi$	$\iff \exists w' \prec w. (r, \pi, w', \mathcal{V} \models \psi) \wedge$ $\quad \forall w''. w' \prec w'' \preceq w \rightarrow (r, \pi, w'', \mathcal{V} \models \varphi)$
$r, \pi, w, \mathcal{V} \models \mathbf{Y} \varphi$	$\iff \exists w' \triangleleft w. r, \pi, w', \mathcal{V} \models \varphi$

Fig. 4. Semantics of formulas

is true at some time t whenever there exists a later time t' such that φ is true in the interval $[t, t')$ and ψ is true at t' .

As explained below, some rules are only valid when formulas are annotated with *non-empty annotations*. A non-empty annotation is either of the form $\bullet$, or t , or an interval as long as it is non-empty, e.g., of the form $[t_1, t_2]$ if $t_1 \preceq t_2$. Therefore, to state such rules, TPTL-DIST also features sequents of the form $\Gamma \vdash \mathbf{a}$, stating that $\mathbf{a}$ is a non-empty annotation in the context Γ .

The semantics of annotated formulas is defined as follows $\clubsuit$:

$$\begin{aligned}
r, \pi, \mathcal{V} \models \varphi^\bullet &\iff r, \pi, 0, \mathcal{V} \models \varphi & r, \pi, \mathcal{V} \models \varphi^t &\iff r, \pi, \llbracket t \rrbracket_{\mathcal{V}}, \mathcal{V} \models \varphi \\
r, \pi, \mathcal{V} \models \varphi^{[t_1, t_2]} &\iff \forall w. \llbracket t_1 \rrbracket_{\mathcal{V}} \preceq w \preceq \llbracket t_2 \rrbracket_{\mathcal{V}} \rightarrow r, \pi, w, \mathcal{V} \models \varphi \\
&\text{and similarly for the other intervals}
\end{aligned}$$

The semantics of (1) a list of hypotheses Γ of the form $h_1, \dots, h_n$, (2) a sequent S of the form $\Gamma \vdash \varphi^{\mathbf{a}}$, and (3) a rule R with premises $S_1, \dots, S_n$ and conclusion S , are defined as follows $\clubsuit$:

$$\begin{aligned}
r, \pi, \mathcal{V} \models \Gamma &\iff \forall i \in [1, n]. r, \pi, \mathcal{V} \models h_i \\
r, \pi \models S &\iff \forall \mathcal{V}(\Gamma, \mathbf{a}, \varphi). r, \pi, \mathcal{V} \models \Gamma \rightarrow r, \pi, \mathcal{V} \models \varphi^{\mathbf{a}} \\
r, \pi \models R &\iff r, \pi \models S_1 \rightarrow \dots \rightarrow r, \pi \models S_n \rightarrow r, \pi \models S
\end{aligned}$$

A rule R is valid iff $r, \pi \models R$ for all runs r and interpretations π . As further discussed in Sec. 5, the above semantics is formalized in Agda, and we have proved that TPTL-DIST's rules presented below are valid.

3.5 Sequent Calculus

We now present an exert of TPTL-DIST's rules. See Appx. B for the full list.

Propositional Logic Rules The propositional logic rules are all standard $\clubsuit$:

$$\begin{array}{c} \frac{\Gamma, \Delta \vdash \mathbf{a}_1}{\Gamma, \perp^{\mathbf{a}_1}, \Delta \vdash \varphi^a} \perp\text{L} \quad \frac{\Gamma \vdash \varphi^a \quad \Gamma \vdash \psi^a}{\Gamma \vdash \varphi \wedge \psi^a} \wedge\text{R} \quad \frac{\Gamma, \varphi^a, \psi^a, \Delta \vdash \delta^{\mathbf{a}_1}}{\Gamma, \varphi \wedge \psi^a, \Delta \vdash \delta^{\mathbf{a}_1}} \wedge\text{L} \\[10pt] \frac{\Gamma \vdash \varphi^a}{\Gamma \vdash \varphi \vee \psi^a} \vee\text{R}_1 \quad \frac{\Gamma \vdash \psi^a}{\Gamma \vdash \varphi \vee \psi^a} \vee\text{R}_2 \quad \frac{\Gamma, \varphi^t, \Delta \vdash \delta^a \quad \Gamma, \psi^t, \Delta \vdash \delta^a}{\Gamma, \varphi \vee \psi^t, \Delta \vdash \delta^a} \vee\text{L} \\[10pt] \frac{\Gamma, \varphi^t \vdash \psi^t}{\Gamma \vdash \varphi \rightarrow \psi^t} \rightarrow\text{R} \quad \frac{\Gamma, \Delta \vdash \varphi^t \quad \Gamma, \psi^t, \Delta \vdash \delta^a}{\Gamma, \varphi \rightarrow \psi^t, \Delta \vdash \delta^a} \rightarrow\text{L} \quad \frac{}{\Gamma, \varphi^a \vdash \varphi^a} \text{var} \end{array}$$

Note that the disjunction in the $\vee\text{L}$ rule cannot be annotated with an interval, since the semantics of intervals is a $\forall$, and $\forall$ does not distribute over $\vee$. Similarly, $\rightarrow\text{R}$ cannot be annotated with intervals either.

Non Empty Annotations Another issue arises with $\perp\text{L}$, which would not hold without the premise $\Gamma, \Delta \vdash \mathbf{a}_1$. For example, the instance $\perp^{\mathbf{a}_1} \vdash \perp^*$ is semantically false when $\mathbf{a}_1$ is an empty interval because $\perp^{\mathbf{a}_1}$ is vacuously true. TPTL-DIST features the following rules to prove that annotations are non-empty $\clubsuit$:

$$\begin{array}{c} \frac{\Gamma \vdash t_1 \sqsubseteq t_2}{\Gamma \vdash [t_1, t_2]} \quad \frac{\Gamma \vdash t_1 \sqsubset t_2}{\Gamma \vdash [t_1, t_2)} \quad \frac{\Gamma \vdash t_1 \sqsubset t_2}{\Gamma \vdash (t_1, t_2]} \quad \frac{\Gamma \vdash t_1 \sqsubset t \quad \Gamma \vdash t \sqsubset t_2}{\Gamma \vdash (t_1, t_2)} \\[10pt] \frac{}{\Gamma \vdash t} \quad \frac{}{\Gamma \vdash \cdot} \end{array}$$

Temporal Logic Rules We now present the temporal rules, which illustrate key aspects of our calculus, where v_t is a “fresh” variable $\clubsuit$:

$$\begin{array}{c} \frac{\Gamma \vdash t \triangleleft t_1 \quad \Gamma \vdash \varphi^{t_1}}{\Gamma \vdash \bigcirc \varphi^t} \bigcirc\text{R} \quad \frac{\Gamma \vdash t \sqsubseteq t_1 \quad \Gamma \vdash \psi^{t_1} \quad \Gamma \vdash \varphi^{[t, t_1)}}{\Gamma \vdash \varphi \mathbf{U} \psi^t} \mathbf{UR} \\[10pt] \frac{\Gamma, t \triangleleft v_t, \varphi^{v_t}, \Delta \vdash \delta^a}{\Gamma, \bigcirc \varphi^t, \Delta \vdash \delta^a} \bigcirc\text{L} \quad \frac{\Gamma, t \sqsubseteq v_t, \psi^{v_t}, \varphi^{[t, v_t)}, \Delta \vdash \delta^a}{\Gamma, \varphi \mathbf{U} \psi^t, \Delta \vdash \delta^a} \mathbf{UL} \end{array}$$

The $\mathbf{UR}$ rule states that for $\varphi \mathbf{U} \psi$ to be true at time t , there must exist a time t_1 greater than t at which ψ holds, and φ holds between t and t_1 . The $\mathbf{UL}$ rule states that if $\varphi \mathbf{U} \psi$ is true at time t then there must be a time v_t (a variable) greater than t at which ψ holds, and φ must be true between t and v_t .

Timed Rules The rules for the comparison and freeze operators are as follows (we only show some of the comparison rules, as they reflect the axioms in Fig. 2) $\clubsuit$:

$$\begin{array}{c} \frac{\Gamma, \varphi[v_t \setminus t]^t, \Delta \vdash \psi^a}{\Gamma, v_t \cdot \varphi^t, \Delta \vdash \psi^a} \cdot\text{L} \quad \frac{\Gamma \vdash \varphi[v_t \setminus t]^t}{\Gamma \vdash v_t \cdot \varphi^t} \cdot\text{R} \\[10pt] \frac{}{\Gamma \vdash t_1 \bullet t_2 \equiv t_2 \bullet t_1^a} \bullet\text{sym} \quad \frac{}{\Gamma, t_1 \sqsubseteq t_2, \Delta \vdash t_1 \sqsubseteq t_2^a} \text{var}\sqsubseteq \end{array}$$

Note that the annotation on the hypothesis $t_1 \sqsubseteq t_2$ of the $\text{var}\sqsubseteq$ rule could be any annotation except an empty interval, unless $\mathbf{a}$ is also an empty interval.

Furthermore, TPTL-DIST's $(\text{ })L$, $(\text{ })L$, $(\text{ })L$, $(\text{ })L$, $(\text{ })R$, $(\text{ })R$, $(\text{ })R$, and $(\text{ })R$ rules allow instantiating intervals, such as the following ones, where v_t is a fresh variable $\clubsuit$:

$$\frac{\Gamma, \varphi^t, \Delta \vdash \psi^a \quad \Gamma, \Delta \vdash t_1 \sqsubseteq t \quad \Gamma, \Delta \vdash t \sqsubseteq t_2}{\Gamma, \varphi^{[t_1, t_2]}, \Delta \vdash \psi^a} \text{[]L} \quad \frac{\Gamma, t_1 \sqsubseteq v_t, v_t \sqsubseteq t_2 \vdash \phi^{v_t}}{\Gamma \vdash \phi^{[t_1, t_2]}} \text{[]R}$$

$$\frac{\Gamma, \varphi^t, \Delta \vdash \psi^a \quad \Gamma, \Delta \vdash t_1 \sqsubseteq t \quad \Gamma, \Delta \vdash t \sqsubseteq t_2}{\Gamma, \varphi^{[t_1, t_2)}, \Delta \vdash \psi^a} \text{[]L} \quad \frac{\Gamma, t_1 \sqsubseteq v_t, v_t \sqsubseteq t_2 \vdash \phi^{v_t}}{\Gamma \vdash \phi^{[t_1, t_2)}} \text{[]R}$$

Induction Furthermore, assuming that worlds satisfy the induction axiom in Fig. 2, the following rule holds, assuming that v_t does not occur free in Γ $\clubsuit$:

$$\frac{\Gamma, v_t \cdot \varphi^{[0, v_t)} \vdash \varphi^{v_t}}{\Gamma \vdash \varphi^{v_t}} \text{ind}$$

Note the use of the freeze operator in the induction hypothesis (IH) $v_t \cdot \varphi^{[0, v_t)}$. It is necessary because not only is v_t the time at which φ is stated to hold, but v_t can also occur free in φ . If the IH were $\varphi^{[0, v_t)}$ (without the freeze operator), then we would be assuming that for any time t in $[0, v_t)$, the formula φ holds at t , but where the variable v_t in φ is still v_t . However, we want the variable v_t to be t too, which the freeze operator achieves by binding v_t 's occurrences in φ .

Another naive attempt at stating this induction hypothesis is $\mathbf{Y}(\blacksquare v_t \cdot \varphi)^{v_t}$. However, this fails in capturing the induction hypothesis because $\mathbf{Y}$ requires the existence of a world preceding v_t , which might not exist. To state the induction hypothesis in that fashion, TPTL-DIST would need to include an alternative $\mathbf{Y}$ operator, say $\mathbf{Y}_?$, with semantics $\forall w' \prec w. (r, \pi, w', \mathcal{V} \models \varphi)$. We instead rely on TPTL-DIST's intervals as shown above. Finally, the hypothesis would have to be $\mathbf{Y}_?(\blacksquare v_t \cdot \varphi)^{v_t}$ and not $v_t \cdot \mathbf{Y}(\blacksquare \varphi)^{v_t}$, which has a different semantics.

Classical Rules TPTL-DIST features a *lem* rule to make classical reasoning explicit without multiple conclusions $\clubsuit$:

$$\frac{}{\Gamma \vdash \varphi \vee \neg \varphi^a} \text{lem}$$

Derived Rules Using the above rules, in particular **UR** and **UL**, right and left rules can be derived for $\Box$ and $\Diamond$, and their past, bounded, and strict versions. These rules will in particular be critical in proving properties of real-time systems as illustrated with a simple example in Sec. 3.6. In all the rules below, v_t stands for a “fresh” variable. For example, the following rules hold for $\Box$, the bounded version of $\Diamond$, and $\blacktriangleleft$ $\clubsuit$:

$$\frac{\Gamma, t \sqsubseteq v_t \vdash \varphi^{v_t}}{\Gamma \vdash \Box \varphi^t} \Box R \quad \frac{\Gamma, \varphi^{t_2}, \Delta \vdash \psi^t \quad \Gamma, \Delta \vdash t_1 \sqsubseteq t_2}{\Gamma, \Box \varphi^{t_1}, \Delta \vdash \psi^t} \Box L$$

$$\frac{\Gamma \vdash \varphi^{t_1} \quad \Gamma \vdash t_1 \sqsubseteq t}{\Gamma \vdash \blacktriangleleft \varphi^t} \blacktriangleleft R \quad \frac{\Gamma \vdash \varphi^{t_1} \quad \Gamma \vdash t \sqsubseteq t_1 \quad \Gamma \vdash t_1 \sqsubseteq t_0 \bullet t}{\Gamma \vdash \Diamond_{t_0} \varphi^t} \Diamond_{-} R$$

$$\frac{\Gamma, \varphi^{v_t}, v_t \sqsubseteq t_1, \Delta \vdash \psi^t}{\Gamma, \blacktriangleleft \varphi^{t_1}, \Delta \vdash \psi^t} \blacktriangleleft L \quad \frac{\Gamma, \varphi^{v_t}, t_1 \sqsubseteq v_t, v_t \sqsubseteq t_0 \bullet t_1, \Delta \vdash \psi^t}{\Gamma, \Diamond_{t_0} \varphi^{t_1}, \Delta \vdash \psi^t} \Diamond_{-} L$$

Soundness

Theorem 1. *TPTL-DIST is sound w.r.t. the semantics presented in Sec. 3.3 and Sec. 3.4, i.e., all the rules presented above are valid.*

3.6 Example

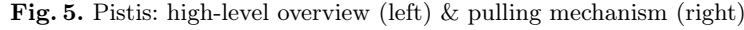
Sec. 3.2 introduced a simple forwarding example, where agents forward the messages they receive in a synchronous network, guaranteeing that any message sent to a single agent is received by all agents by some time bound. As sketched below, TPTL-DIST allows proving the following property (see Sec. C for full details):

Lemma 1. *This following holds in all models $\clubsuit$: $(Eq. 1)^t, (Eq. 2)^t \vdash Eq. 3^t$.*

Proof. Assume $(Eq. 1)^t$ and $(Eq. 2)^t$, agents a , b , and c , and data d , and prove that $\Box(\text{send}(a, d, [b]) \rightarrow \Diamond_{2T}\text{recv}(c, d, b))$ holds at t . By $\Box R$, it is enough to prove that $\Diamond_{2T}\text{recv}(c, d, b)$ holds at some time v_t , assuming $t \sqsubseteq v_t$ and $\text{send}(a, d, [b])^{v_t}$. In particular, from $\text{send}(a, d, [b])^{v_t}$ and $(Eq. 1)^t$, using $\Box L$, we get to further assume that $\Diamond_T(b \in [b] \rightarrow \text{recv}(b, d, a))^{v_t}$. Now by $\Diamond_- L$, we can refine this assumption to $\text{recv}(b, d, a)^{v'_t}$ for some v'_t such that $v_t \sqsubseteq v'_t$ and $v'_t \sqsubseteq T \bullet v_t$. From $\text{recv}(b, d, a)^{v'_t}$ and $(Eq. 2)^t$, using $\Box L$, we get to further assume $\text{send}(b, d, [c])^{v'_t}$. Once again, using $(Eq. 1)^t$ and $\Box L$ on that hypothesis, we get to assume $\Diamond_T(c \in [c] \rightarrow \text{recv}(c, d, b))^{v'_t}$, and using $\Diamond_- L$, we can refine this assumption to $\text{recv}(c, d, b)^{v''_t}$ for some v''_t such that $v'_t \sqsubseteq v''_t$ and $v''_t \sqsubseteq T \bullet v'_t$. Finally, we conclude using $\Diamond_- R$, from the fact that $v_t \sqsubseteq v''_t$ and $v''_t \sqsubseteq 2T \bullet v_t$. $\square$

4 Real-Time Distributed Systems**4.1 The Pistis Real-Time Reliable Broadcast Protocol**

Our main use case is the Pistis real-time Byzantine-resilient reliable broadcast protocol [16]. A broadcast protocol allows messages sent by (correct) nodes in a distributed system to be eventually received (or delivered) by the other (correct) nodes when the network conditions are “good enough”. We are in particular interested here in scenarios where nodes can be faulty, where faults can range from benign crashes to more critical adversarial attacks. This fault model is called the *Byzantine* fault model [38, Sec.2.2.6], in which faulty nodes behave arbitrarily. We further assume that nodes can be authenticated, usually achieved through digital signatures. All messages are signed by their senders, meaning that correct nodes cannot be impersonated. Many Byzantine-resilient broadcast protocols exist, providing different properties. In particular, Byzantine consistent broadcast [38, Sec.3.10], guarantees that if a correct node broadcasts a message then that message will be delivered by all other correct nodes. This however does not cover the scenario where the broadcasting node is faulty, in which case only some of the other correct nodes might deliver the message. Byzantine reliable



Many Byzantine reliable broadcast algorithms have been developed, such as Bracha’s algorithm [39], which requires $3f+1$ nodes to tolerate up to f Byzantine faults, and makes use of two message transmission phases (often called *echo* and *ready*) to guarantee the above properties. More recently, algorithms have been designed that further guarantee a timeliness property [16, 40]. For example, Pistis [16] builds on top of Bracha’s algorithm, and further satisfies *Timeliness*, i.e., there exists a known bound Δ such that if a correct node broadcasts a message m at real-time t , then no correct nodes delivers m after $t + \Delta$.

Fig. 5 illustrates the Pistis protocol with 4 nodes, i.e., at most 1 faulty node. Node i broadcasts a message by sending an echo message to all other nodes, which reply by sending their own echo messages to all other nodes. Nodes that

receive $2f+1$ echo messages,⁴ including their own, (or a ready message) proceed by sending a ready message to all other nodes. Any node that has not received $2f+1$ echo messages (or one ready message) T units of time after it has sent its echo message becomes passive. Similarly, any node that has not received $2f+1$ ready messages $2T$ units of time after it has sent its ready message becomes passive. We expand on those T and $2T$ bounds below.

We now introduce Pistis in more detail, describing its main functionalities in terms of pushing and pulling messages, leading to a discussion on how it achieves timeliness, while Sec. 4.2 illustrates how to verify real-time properties of distributed systems using TPTL-DIST by proving a property crucial for timeliness.

Pushing A node that sends a message at time t without receiving $2f+1$ replies by $t+T$, becomes passive. Here, T is a multiple of d , the maximum transmission delay, picked by the system administrator depending on the quality of the network. Assuming synchrony, it would be guaranteed that all messages sent at t are received by correct nodes by $t+d$. However, assuming probabilistic synchrony, we can only derive the weaker “quorum” synchrony assumption guaranteeing that if a correct node sends a message at time t , then a “quorum” of $2f+1$ nodes are known to have received it by time $t+T$. This property can be stated as follows, where a “Quorum” is a set of $2f+1$ nodes, $f+1$ of which are therefore correct :

$$\begin{aligned} & \forall p : \text{Node}. \text{correct}(p) \rightarrow \forall m : \text{Msg}. \forall t : \text{Time}. p \text{ sends } m \text{ at } t \rightarrow \\ & \exists Q : \text{Quorum}. \forall q \in Q. \\ & \quad p \text{ receives the acknowledgment within } [t, t+T] \text{ that } q \text{ received } m \end{aligned} \quad (4)$$

Pulling To guarantee timeliness despite messages arriving late, nodes implement the above PoC mechanism, which sends *heartbeat* messages every d units of time to probe the rest of the system and fetch (or *pull*) broadcast messages. A node that does not receive enough replies to its heartbeats becomes passive, as it risks delivering late. Heartbeats are tagged with unique identifiers, and given a heartbeat sent by node i , tagged with identifier k , denoted $\text{HB}(k)$ in Fig. 5, and sent at time t , $2f+1$ replies to $\text{HB}(k)$ are expected by $t+T$ for i to stay active and therefore correct. PoC guarantees that if a quorum of nodes sends a message within $[t, t+T]$, then all correct nodes will receive that message within $[t+T, t+2T]$, thereby preventing nodes from staying active while not delivering:

$$\begin{aligned} & \forall m : \text{Msg}. \forall t : \text{Time}. \forall Q : \text{Quorum}. (\forall q \in Q. q \text{ sends } m \text{ within } [t, t+T]) \rightarrow \\ & \forall p : \text{Node}. \text{correct}(p) \rightarrow p \text{ receives } m \text{ within } [t+T, t+2T] \end{aligned} \quad (5)$$

A simple way to guarantee Eq. 5 is for nodes to reply to heartbeats with the messages they have sent during the last $2T$ time interval. A $2T$ interval is necessary because a node (i_1 in Fig. 5) in Q sending m at t (one end of the $[t, t+T]$ interval) piggybacks m to its replies to heartbeats during the interval $[t, t+2T]$, while a node (i_3 in Fig. 5) sending m at $t+T$ (the other end of the $[t, t+T]$ interval) piggybacks m during the interval $[t+T, t+3T]$, thereby intersecting on the

⁴ Since there are $3f+1$ nodes, f of which might be faulty, nodes can only expect to receive messages from $2f+1$ nodes.

$$\begin{aligned}
 \text{bounded delivery: } & \Box \forall i : \mathbf{AGENT}. \forall d : \mathbf{DATA}. \text{correct}(i) \rightarrow \text{internal}(i, d) \rightarrow \\
 & \quad \exists B : \mathbf{AGENTS}. \text{size}(B, 2f+1) \wedge \\
 & \quad \quad \Diamond_T \Diamond \forall j : \mathbf{AGENT}. j \in B \rightarrow \text{correct}(j) \rightarrow \Diamond_T \text{internal}(j, d) \\
 \text{unbounded delivery: } & \Box \forall i : \mathbf{AGENT}. \forall d : \mathbf{DATA}. \text{correct}(i) \rightarrow \text{internal}(i, d) \rightarrow \\
 & \quad \exists B : \mathbf{AGENTS}. \text{size}(B, 2f+1) \wedge \\
 & \quad \quad \forall j : \mathbf{AGENT}. j \in B \rightarrow \text{correct}(j) \rightarrow \Diamond_T \Diamond \text{internal}(j, d) \\
 \text{stay faulty: } & \forall i : \mathbf{AGENT}. \Box (\text{correct}(i) \rightarrow \blacksquare \text{correct}(i))
 \end{aligned}$$

Fig. 6. Bounded and unbounded delivery in TPTL-DIST

interval $[t+T, t+2T]$. As Fig. 5 illustrates, these two mechanisms (sending and replying to heartbeats) guarantee that, during the $[t+T, t+2T]$ interval, correct (non-passive) nodes (i in Fig. 5) obtain messages sent by quorums of nodes. This holds because one correct node (i_2 in Fig. 5) is necessarily at the intersection of the $2f+1$ nodes sending replies to $\text{HB}(k)$ and the $2f+1$ nodes piggybacking m .

Properties The above components guarantee timeliness, since by prop. 4, if p is correct and broadcasts a message m at time t , then by time $t_d \leq t+T$, it receives the acknowledgments that m was received by $2f+1$ nodes. It can then start delivering and sending corresponding ready messages. Again by prop. 4, by time t_d+T , p receives the acknowledgment that this ready message was received by $2f+1$ nodes. We then prove by induction on time t_d , that there is a collection B of $2f+1$ nodes such that all correct nodes in B started delivering m (and sending corresponding ready messages) during some interval $[t_{d'}, t_{d'}+T]$ some time prior to t_d+T . Therefore, their PoC components will piggyback those ready messages to heartbeats (ie., to pull requests) during the interval $[t_{d'}+T, t_{d'}+2T]$. By prop. 5, all correct nodes receive m during that interval.

Therefore, a key point in this proof is that, as captured by *bounded delivery* in Fig. 6 , if a correct node delivers at time t , then there exists a collection B of $2f+1$ nodes such that some time t' prior to $t+T$, all correct nodes in B have delivered by $t'+T$. This follows from the fact that, as captured by the *unbounded delivery* TPTL-DIST formula in Fig. 6, if a correct node delivers at time t , then there exists a collection of $2f+1$ nodes that have delivered some time prior to $t+T$, a consequence of prop. 4. Sec. 4.2 shows that *unbounded delivery* implies *bounded delivery*, assuming that faulty nodes cannot become correct.

4.2 Proof of Bounded Delivery

We now show how to prove that *bounded delivery* follows from *unbounded delivery* (see Fig. 6), which is key in proving Pistis' timeliness property.

Lemma 2. *The following sequent holds in all models :*

$$\text{unbounded delivery}^0, \text{stay faulty}^0 \vdash \text{bounded delivery}^0$$

Proof. Using $\Box R$, we have to prove $unbounded\ delivery \vdash C^{v_t}$, where v_t is a fresh variable, and C is the formula under the $\Box$ in $bounded\ delivery$. Using the induction rule from Sec. 3.5, we obtain the additional assumption $v_t \cdot C^{[0, v_t]}$.

We then assume some correct agent i and piece of data d that “happened” at i at time v_t , and have to exhibit a quorum of agents B , such that at some time t before $T \bullet v_t$, d “happened” at all correct agents in B by time $T \bullet t$ (as stated by $bounded\ delivery$). Instantiating $unbounded\ delivery$ to v_t using $\Box L$, we know that there exists a quorum of agents B , such that d “happened” at all correct agents in B prior to $T \bullet v_t$. Using the classical *lem* rule, we go by cases on whether $\Diamond_T \Diamond \forall j : \mathbf{AGENT}.j \in B \rightarrow \mathbf{correct}(j) \rightarrow \Diamond_T \mathbf{internal}(j, d)$ holds at time v_t , concluding directly if it does.

Let us now assume that the above formula does not hold. Using classical reasoning, we derive that $\Box_T \blacksquare \exists j : \mathbf{AGENT}.j \in B \wedge \mathbf{correct}(j) \wedge \Box_T \neg \mathbf{internal}(j, d)$ holds at v_t . Instantiating both $\Box$ and $\blacksquare$ with the current time v_t , we get a correct agent j in B such that $\Box_T \neg \mathbf{internal}(j, d)$ holds at v_t (note that the assumption that j is correct is at v_t). From $unbounded\ delivery$, we also get that $\Diamond_T \Diamond \mathbf{internal}(j, d)$ holds at v_t . It then follows that $\blacktriangleleft \mathbf{internal}(j, d)$ holds at v_t too. Using $\blacktriangleleft L$, it follows that $\mathbf{internal}(j, d)$ holds at $v'_t \sqsubseteq v_t$.

Using $\Box L$, we instantiate the induction hypothesis with v'_t , j , and d , and obtain a quorum of agents B' such that $\Diamond_T \Diamond \forall j : \mathbf{AGENT}.j \in B' \rightarrow \mathbf{correct}(j) \rightarrow \Diamond_T \mathbf{internal}(j, d)$ holds at v'_t (we then have to prove that j is correct at $v'_t \sqsubseteq v_t$, which holds thanks to *stay faulty*). We conclude by instantiating the conclusion with B' , and by showing that if $\Diamond_t \Diamond A$ holds at $t_1 \sqsubseteq t_2$ then it also holds at t_2 $\square$

5 Conclusion

We have introduced TPTL-DIST, a calculus for verifying time-related properties of fault-tolerant distributed systems, extending TPTL + Past with quantifiers and a general comparison operator, featuring a sequent calculus that relies on intervals to precisely capture timing constraints. TPTL-DIST allows formalizing and verifying non-trivial properties of fault-tolerant algorithms, including one of Pistis’ key properties. We now plan on exploring a few directions.

Based on TPTL-DIST, we plan on designing a type system tailored to the unique demands of distributed systems, that would not only allow modeling their properties, but also allow reasoning about running code.

Another direction is to build on the architecture of PRISM to adapt existing model checkers and automated verification techniques to TPTL-DIST. While subsets of TPTL-DIST could be model checked by PRISM, those techniques will need to be adapted to handle TPTL-DIST’s freeze and distributed operators.

Finally, all the results discussed above have been formalized in Agda. In particular, we have proved that the rules in Sec. 3.5 are valid according to TPTL-DIST’s semantics. Using those proofs, we have started building a verified proof checker for TPTL-DIST in Agda, which opens the door to further automation. In addition, we proved that a large subset of TPTL-DIST has a decidable semantics, which we will integrate in a formally verified automated proof engine.

References

- [1] R. Alur and T. A. Henzinger. “A really temporal logic”. In: *Journal of the ACM* 41.1 (Jan. 1994), pp. 181–203.
- [2] C. Baier and J. Katoen. *Principles of model checking*. MIT Press, 2008.
- [3] M. Kwiatkowska, G. Norman, and D. Parker. “PRISM 4.0: Verification of Probabilistic Real-time Systems”. In: *Proc. 23rd International Conference on Computer Aided Verification (CAV’11)*. Vol. 6806. LNCS. Springer, 2011, pp. 585–591.
- [4] L. Lamport. *Specifying systems: the TLA+ language and tools for hardware and software engineers*. Boston: Addison-Wesley, 2003.
- [5] L. Lamport. *Real Time is Really Simple*. Tech. rep. MSR-TR-2005-30. Mar. 2005, p. 72.
- [6] D. Cousineau, D. Doligez, L. Lamport, S. Merz, D. Ricketts, and H. Vanzetto. *TLA+ Proofs*. 2012. arXiv: [1208.5933 \[cs.SE\]](#).
- [7] G. Norman, D. Parker, and X. Zou. “Verification and Control of Partially Observable Probabilistic Real-Time Systems”. In: *Formal Modeling and Analysis of Timed Systems - 13th International Conference, FORMATS 2015*. Vol. 9268. Lecture Notes in Computer Science. Springer, 2015, pp. 240–255.
- [8] M. Oxford, D. Parker, and M. Ryan. “Quantitative Verification of Certificate Transparency Gossip Protocols”. In: *8th IEEE Conference on Communications and Network Security, CNS 2020, Avignon, France, June 29 - July 1, 2020*. IEEE, 2020, pp. 1–9.
- [9] T. Lu, S. Merz, and C. Weidenbach. “Towards Verification of the Pastry Protocol Using TLA⁺”. In: 2011, pp. 244–258.
- [10] R. Joshi, L. Lamport, J. Matthews, S. Tasiran, M. R. Tuttle, and Y. Yu. “Checking Cache-Coherence Protocols with TLA⁺”. In: *Formal Methods in System Design* 22.2 (2003), pp. 125–131.
- [11] W. J. Bolosky, J. R. Douceur, and J. Howell. “The Farsite project: a retrospective”. In: *Operating Systems Review* 41.2 (2007), pp. 17–26.
- [12] C. Newcombe. “Why Amazon Chose TLA⁺”. In: 2014, pp. 25–39.
- [13] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardouff. “How Amazon web services uses formal methods”. In: *Commun. ACM* 58.4 (2015), pp. 66–73.
- [14] S. Chand, Y. A. Liu, and S. D. Stoller. “Formal Verification of Multi-Paxos for Distributed Consensus”. In: *FM 2016: Formal Methods - 21st International Symposium*. Vol. 9995. 2016, pp. 119–136.
- [15] S. Gao, B. Zhan, D. Liu, X. Sun, Y. Zhi, D. N. Jansen, and L. Zhang. “Formal Verification of Consensus in the Taurus Distributed Database”. In: *Formal Methods*. 2021, pp. 741–751.
- [16] D. Kozhaya, J. Decouchant, V. Rahli, and P. E. Veríssimo. “PISTIS: An Event-Triggered Real-Time Byzantine-Resilient Protocol Suite”. In: *IEEE Transactions on Parallel and Distributed Systems* 32.9 (2021), pp. 2277–2290.

- [17] M. Volpe. “Labeled natural deduction for temporal logics”. PhD thesis. University of Verona, Italy, 2010.
- [18] H. Kopetz. *Real-Time Systems: Design Principles for Distributed Embedded Applications*. 1st. USA: Kluwer Academic Publishers, 1997.
- [19] H. Kopetz. “Why a Global Time is Needed in a Dependable SoS”. In: *CoRR* abs/1404.6772 (2014). arXiv: [1404.6772](#).
- [20] H. Hansson and B. Jonsson. “A Logic for Reasoning about Time and Reliability”. In: *Formal Aspects of Computing* 6.5 (1994), pp. 512–535.
- [21] A. Bianco and L. de Alfaro. “Model Checking of Probabilistic and Nondeterministic Systems”. In: *Proc. 15th Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS’95)*. Vol. 1026. LNCS. Springer, 1995, pp. 499–513.
- [22] A. Aziz, K. Sanwal, V. Singhal, and R. Brayton. “Verifying Continuous Time Markov Chains”. In: *Proc. 8th International Conference on Computer Aided Verification (CAV’96)*. Vol. 1102. LNCS. Springer, July 1996, pp. 269–276.
- [23] C. Baier, J.-P. Katoen, and H. Hermanns. “Approximate Symbolic Model Checking of Continuous-Time Markov Chains”. In: *Proc. 10th International Conference on Concurrency Theory (CONCUR’99)*. Vol. 1664. LNCS. Eindhoven: Springer, Aug. 1999, pp. 146–161.
- [24] L. Bozzelli, A. Murano, and L. Sorrentino. “Alternating-time temporal logics with linear past”. In: *Theoretical Computer Science* 813 (Apr. 2020), pp. 199–217.
- [25] R. Fagin, J. Y. Halpern, Y. Moses, and M. E. Vardi. *Reasoning about knowledge*. Cambridge, Mass: MIT Press, 1995.
- [26] F. Wehrung. “Injective positively ordered monoids I”. In: *Journal of Pure and Applied Algebra* 83.1 (1992), pp. 43–82.
- [27] T. FRITZ. “Resource convertibility and ordered commutative monoids”. In: *Mathematical Structures in Computer Science* 27.6 (2017), pp. 850–938.
- [28] R. Kuznets, L. Prosperi, U. Schmid, and K. Fruzsa. “Epistemic Reasoning with Byzantine-Faulty Agents”. In: *Frontiers of Combining Systems*. Vol. 11715. Cham: Springer International Publishing, 2019, pp. 259–276.
- [29] A. Baltag, B. Renne, and S. Smets. “The logic of justified belief, explicit knowledge, and conclusive evidence”. In: *Annals of Pure and Applied Logic* 165.1 (Jan. 2014), pp. 49–81.
- [30] A. Baltag, L. S. Moss, and S. Solecki. “The Logic of Public Announcements, Common Knowledge, and Private Suspicions (extended abstract)”. In: ().
- [31] A. Bolotov, A. Basukoski, O. Grigoriev, and V. Shangin. “Natural Deduction Calculus for Linear-Time Temporal Logic”. In: *Logics in Artificial Intelligence*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 56–68.
- [32] A. Bolotov, O. Grigoriev, and V. Shangin. “Automated Natural Deduction for Propositional Linear-Time Temporal Logic”. In: *14th International*

- Symposium on Temporal Representation and Reasoning (TIME'07)*. 2007, pp. 47–58.
- [33] A. Masini, L. Viganò, and M. Volpe. “A History of Until”. In: *Electron. Notes Theor. Comput. Sci.* 262 (May 2010), pp. 189–204.
 - [34] D. M. Gabbay. *Labelled Deductive Systems*. Clarendon Press, 1996.
 - [35] A. K. Simpson. “The proof theory and semantics of intuitionistic modal logic”. PhD thesis. University of Edinburgh, UK, 1994.
 - [36] D. Basin, C. Caleiro, J. Ramos, and L. Viganò. “Labelled Tableaux for Distributed Temporal Logic”. In: *Journal of Logic and Computation* 19.6 (July 2009), pp. 1245–1279. eprint: <https://academic.oup.com/logcom/article-pdf/19/6/1245/2805892/exp022.pdf>.
 - [37] L. Viganò and M. Volpe. *Labeled Natural Deduction Systems for a Family of Tense Logics*. 2008. arXiv: [0803.3187](https://arxiv.org/abs/0803.3187) [cs.LO].
 - [38] C. Cachin, R. Guerraoui, and L. Rodrigues. *Introduction to Reliable and Secure Distributed Programming*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011.
 - [39] G. Bracha. “Asynchronous Byzantine Agreement Protocols”. In: *Inf. Comput.* 75.2 (1987), pp. 130–143.
 - [40] D. Kozhaya, J. Decouchant, and P. Esteves-Verissimo. “RT-ByzCast: Byzantine-Resilient Real-Time Reliable Broadcast”. In: *IEEE Trans. Comput.* 68.3 (2019), pp. 440–454.
 - [41] G. L. Lann. “Distributed Systems - Towards a Formal Approach”. In: *Information Processing, Proceedings of the 7th IFIP Congress 1977, Toronto, Canada, August 8-12, 1977*. North-Holland, 1977, pp. 155–160.
 - [42] E. J. H. Chang and R. Roberts. “An Improved Algorithm for Decentralized Extrema-Finding in Circular Configurations of Processes”. In: *Commun. ACM* 22.5 (1979), pp. 281–283.

A Further Examples

In addition to the forwarding (FW) example provided in Sec. 3.2, we further illustrate here aspects of TPTL-DIST through two other simple examples: leader election in a ring (LER) in Sec. A.1, and Byzantine consistent broadcast (BCB) in Sec. A.2. All three examples illustrate how to capture message passing patterns in distributed systems, where agents exchange information by sending/receiving messages. Furthermore, while the LER and BCB examples involve a logical, qualitative notion of time, the FW example involves a physical, quantitative notion of time thanks to formulas of the form $\Diamond_t \varphi$. Finally, the LER example involves a typical inductively proven property, while BCB involves some typical pattern, where agents can only make progress when they have received enough messages from the other agents in the system.

A.1 Leader Election in a Ring

The leader election in a ring problem involves selecting a single leader from a group of agents organized in a ring. Each agent has a unique identifier (id), and

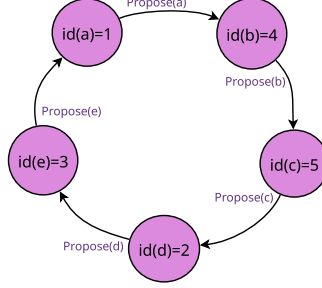


Fig. 7. Leader Election in a Ring

the goal is to elect the agent with the highest id. Agents do not have knowledge of the entire ring, and messages can only be passed to the adjacent node according to a fixed (e.g., clockwise) direction. Leader election is crucial for coordination in distributed systems, such as for database replication or time synchronization.

A key property that the algorithm must fulfill is *liveness*, which guarantees that all agents eventually receive the outcome of the election, i.e., the identity of the leader:

$$\forall a : \mathbf{AGENT}. \exists b, c : \mathbf{AGENT}. @_0 \Diamond \text{recv}(a, \text{Elected}(c), b) \quad (6)$$

The $@$ operator is used to state that a formula is true at a specific point in time: $@_t \varphi \equiv x \cdot \varphi \wedge x \equiv t$. Both $\text{Propose}(a)$ and $\text{Elected}(a)$ are pieces of data capturing the fact that a was proposed to be the leader, and elected to be the leader, respectively.

We now present a formalization of the classical ring election algorithm [41, 42] in TPTL-DIST. We assume no failures, a synchronous system, a binary predicate $>$ on agents, where $a > b$ captures the fact that a has a higher identifier than b , and a function $\text{nhbr} : \mathbf{AGENT} \rightarrow \mathbf{AGENT}$ returning the neighbour of each agent in the ring.

Initialization: Each agent a suggests itself as the leader, sending its id in a proposal to its neighbour (See Fig. 7):

$$\forall a : \mathbf{AGENT}. @_0 \text{send}(a, \text{Propose}(a), \{\text{nhbr}(a)\}) \quad (7)$$

Proposal Propagation: When an agent a receives an $\text{Propose}(c)$ message, it compares c 's id to its own id. If c 's id is greater than a 's, a simply forwards the message to its neighbour, and if c 's id is smaller than a 's, a forwards its own id to its neighbour:

$$\begin{aligned} \forall a, b, c : \mathbf{AGENT}. \Box (\text{recv}(a, \text{Propose}(c), b) \rightarrow \\ (c > a \rightarrow \text{send}(a, \text{Propose}(c), \{\text{nhbr}(a)\})) \wedge \\ (a > c \rightarrow \text{send}(a, \text{Propose}(a), \{\text{nhbr}(a)\}))) \end{aligned} \quad (8)$$

Leader Determination: If a receives a proposal containing its own id, it means that a has the highest id. It then sends an $\text{Elected}(a)$ message around

the ring to announce its election as leader.

$$\forall a, b : \mathbf{AGENT}. \Box(\text{recv}(a, \text{Propose}(a), b) \rightarrow \text{send}(a, \text{Elected}(a), \{\text{nhbr}(a)\})) \quad (9)$$

Leader Propagation: When an agent a receives an $\text{Elected}(b)$ message, it knows that b is the leader, and forwards the message to its neighbour (unless a is b):

$$\forall a, b, c : \mathbf{AGENT}. \quad (10) \\ \Box(\text{recv}(a, \text{Elected}(b), c) \rightarrow b > a \rightarrow \text{send}(a, \text{Elected}(b), \{\text{nhbr}(a)\}))$$

The algorithm ends when the leader received an Elected message with its own id.

Liveness (election terminates) is proved by induction on time using the following invariant, which states that at time x , if a has not yet received a message informing it that it is the leader (the leader operator), then it must be receiving the highest id among its x predecessors in the ring (the rcvMaxId operator), where $\text{max}(a, x)$ is the agent with the highest id among the x predecessors of a in the ring (it returns a when x is 0):

$$\forall a : \mathbf{AGENT}. \bigcirc \Box(x \cdot \neg \text{leader}(a) \rightarrow \text{rcvMaxId}(a, \text{max}(a, x))) \quad (11)$$

where

$$\begin{aligned} \text{leader}(a) &\equiv \exists c : \mathbf{AGENT}. \blacklozenge \text{recv}(a, \text{Propose}(a), c) \\ \text{rcvMaxId}(a, b) &\equiv \exists c : \mathbf{AGENT}. \text{recv}(a, \text{Propose}(b), c) \end{aligned}$$

This invariant allows concluding that the election phase terminates after N steps, where N is the number of agents in the ring, with all agents having received the highest agent/leader, including the leader itself. Eq. 9 can then be used for the leader to start the final phase and Eq. 10 guarantees that the Elected message will go around the ring back to the leader marking the end of the execution of the algorithm.

We further consider that the agents advance in lockstep, meaning that each message takes exactly one unit of time to reach its destination:

$$\forall a, b : \mathbf{AGENT}. \forall d : \mathbf{DATA}. \Box(\text{send}(a, d, \{b\}) \rightarrow \bigcirc \text{recv}(b, d, a)) \quad (12)$$

Let us now sketch the proof of the above invariant. In the base case where the time $x = 0$, $\text{max}(a, x) = a$, which satisfies the invariant, because each agent is assumed to have received a vote for itself at initialization as captured by Eq. 7.

For the induction step, assume the property holds for time x . Then, at time $x+1$, the predecessor of agent a , say b , if not yet the leader, has received $\text{max}(b, x)$, which is the agent with the largest id among the x predecessors of b . When b receives this information, it compares its own id with $\text{max}(b, x)$. If $b > \text{max}(b, x)$ or $b < \text{max}(b, x)$, we apply Eq. 8, sending $\text{Propose}(b)$ or $\text{Propose}(\text{max}(b, x))$ to a , respectively. In both cases a receives $\text{max}(a, x+1)$ at time $x+1$ using Eq. 12.

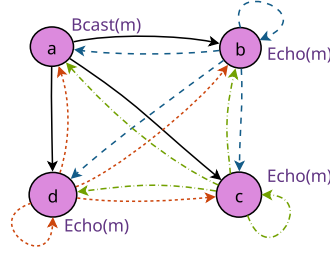


Fig. 8. Byzantine Consistent Broadcast

A.2 The Byzantine Consistent Broadcast

The Byzantine Consistent Broadcast [38, Sec. 3.10] (BCB) is a fundamental broadcast primitive designed for the Byzantine fault model, where faulty agents arbitrarily deviate from their specification, even deliberately trying to prevent an algorithm from achieving its goals. It requires $3f + 1$ agents to tolerate up to f faulty agents.

Every BCB instance has a unique designated sender initiating the broadcast of a message. BCB guarantees the *validity* property, even if the sender is faulty, stating that if a correct agent a broadcasts a message m , then every correct process eventually delivers m . This property is stated as follows in TPTL-DIST, where $\text{Bcast}(m), \text{Echo}(m), \text{Deliver}(m) \in \mathbf{DATA}$ capture the broadcast, echoing, and delivery of a message m (also a piece of data), respectively:

$$\forall a, b : \mathbf{AGENT}. \forall m : \mathbf{DATA}. \text{correct}(a) \rightarrow \text{correct}(b) \rightarrow \quad (13)$$

$$\Box(\text{internal}(a, \text{Bcast}(m)) \rightarrow \Diamond \text{internal}(b, \text{Deliver}(m)))$$

Broadcast: An agent initiates a broadcast through an internal broadcast event of the form $\text{internal}(a, \text{Bcast}(m))$. When initiating a broadcast, the agent sends a broadcast message to all other agents, where N is the collection of agents in the system:

$$\forall a : \mathbf{AGENT}. \forall m : \mathbf{DATA}. \text{correct}(a) \rightarrow \quad (14)$$

$$\Box(\text{internal}(a, \text{Bcast}(m)) \rightarrow \text{send}(a, \text{Bcast}(m), N))$$

Echo: When an agent receives a broadcast message, it then sends an echo message to all other agents:

$$\forall a, b : \mathbf{AGENT}. \forall m : \mathbf{DATA}. \text{correct}(a) \rightarrow \quad (15)$$

$$\Box(\text{recv}(a, \text{Bcast}(m), b) \rightarrow \text{send}(a, \text{Echo}(m), b))$$

Deliver: Agents deliver on receipt of $2f+1$ echoes:

$$\forall a : \mathbf{AGENT}. \Box(\text{correct}(a) \rightarrow \left(\begin{array}{l} \exists B : \mathbf{AGENTS}. \\ \text{size}(B, 2f+1) \wedge \\ \forall b : \mathbf{AGENT}. b \in B \rightarrow \\ \Diamond \text{recv}(a, \text{Echo}(m), b) \end{array} \right) \rightarrow \Diamond \text{internal}(a, \text{Deliver}(m))) \quad (16)$$

Reliable Communication: We assume reliable communication, i.e., any message sent by a correct agent will eventually be received by all other correct agents:

$$\begin{aligned} \forall a : \mathbf{AGENT}. \forall d : \mathbf{DATA}. \forall B : \mathbf{AGENTS}. \quad (17) \\ \Box(\text{correct}(a) \rightarrow \text{send}(a, d, B) \rightarrow \\ \forall b : \mathbf{AGENT}. b \in B \rightarrow \\ \text{correct}(b) \rightarrow \Diamond \text{recv}(b, d, a)) \end{aligned}$$

Fault Threshold: Finally, we assume that there are at least $2f+1$ correct agents out of $3f+1$, where P below is a predicate on agents:

$$\begin{aligned} \Box(\forall b : \mathbf{AGENT}. \text{correct}(b) \rightarrow P(b)) \rightarrow \quad (18) \\ \exists B : \mathbf{AGENTS}. \text{size}(B, 2f+1) \wedge \\ \forall b : \mathbf{AGENT}. b \in B \rightarrow \text{correct}(b) \wedge P(b) \end{aligned}$$

Fig. 8 shows a BCB execution, in a system with $3f+1 = 4$ agents, with at most $f = 1$ Byzantine agent. Agent a broadcasts message m , which is then echoed by all agents (dashed blue lines for b , dashed and dotted green lines for c , and dotted orange lines for d). After received $2f+1 = 3$ echoes from b, c, d , agent b , for example, delivers.

Let us now sketch a proof of the validity property. Assume correct agents a and b such that a initiates a broadcast of m , i.e., $\text{internal}(a, \text{Bcast}(m))$ holds. We need to prove that b eventually delivers m , i.e., $\Diamond \text{internal}(b, \text{Deliver}(m))$. By Eq. 14, a also sends $\text{Bcast}(m)$ to all agents in N . By Eq. 17, all correct agents in N receive $\text{Bcast}(m)$ and by Eq. 15 they send $\text{Echo}(m)$ messages to all other agents in N , including b . Therefore, by Eq. 17, b will receive $\text{Echo}(m)$ from all correct agents. Finally, by Eq. 16 and Eq. 18, agent b delivers m .

B Full Set of Rules

Propositional Logic TPTL-DIST's propositional logic rules are as follows:

$$\begin{array}{c} \frac{\Gamma, \Delta \vdash a_1}{\Gamma, \Delta \vdash a_1} \quad \perp\text{L} \quad \frac{\Gamma \vdash \varphi^a \quad \Gamma \vdash \psi^a}{\Gamma \vdash \varphi \wedge \psi^a} \quad \wedge\text{R} \quad \frac{\Gamma, \varphi^a, \psi^a, \Delta \vdash \delta^{a_1}}{\Gamma, \varphi \wedge \psi^a, \Delta \vdash \delta^{a_1}} \quad \wedge\text{L} \\ \\ \frac{\Gamma \vdash \varphi^a}{\Gamma \vdash \varphi \vee \psi^a} \quad \vee\text{R}_1 \quad \frac{\Gamma \vdash \psi^a}{\Gamma \vdash \varphi \vee \psi^a} \quad \vee\text{R}_2 \quad \frac{\Gamma, \varphi^t, \Delta \vdash \delta^a \quad \Gamma, \psi^t, \Delta \vdash \delta^a}{\Gamma, \varphi \vee \psi^t, \Delta \vdash \delta^a} \quad \vee\text{L} \end{array}$$

$$\frac{}{\Gamma \vdash \top^a} \top R \quad \frac{\Gamma, \varphi^t \vdash \psi^t}{\Gamma \vdash \varphi \rightarrow \psi^t} \rightarrow R \quad \frac{\Gamma, \Delta \vdash \varphi^t \quad \Gamma, \psi^t, \Delta \vdash \delta^a}{\Gamma, \varphi \rightarrow \psi^t, \Delta \vdash \delta^a} \rightarrow L$$

TPTL-DIST also includes the following rules. We often make silent use of the *cut* rule to duplicate an hypothesis whenever using a “left” (or elimination) rule on that hypothesis if we wish to retain it because TPTL-DIST’s “left” rules discard the formulas being eliminated.

$$\frac{\Gamma, \Delta \vdash \varphi^a}{\Gamma, \psi^{a1}, \Delta \vdash \varphi^a} \text{thin} \quad \frac{\Gamma \vdash \psi^{a1} \quad \Gamma, \psi^{a1} \vdash \varphi^a}{\Gamma \vdash \varphi^a} \text{cut}$$

$$\frac{}{\Gamma, \varphi^a \vdash \varphi^a} \text{var} \quad \frac{}{\Gamma \vdash i \in [\dots, i, \dots]^a} \in R$$

Non-Empty Annotations

$$\frac{}{\Gamma \vdash \cdot} \quad \frac{\Gamma \vdash t_1 \sqsubseteq t_2}{\Gamma \vdash [t_1, t_2]} \quad \frac{\Gamma \vdash t_1 \sqsubseteq t_2}{\Gamma \vdash [t_1, t_2]}$$

$$\frac{}{\Gamma \vdash t} \quad \frac{\Gamma \vdash t_1 \sqsubseteq t_2}{\Gamma \vdash (t_1, t_2]} \quad \frac{\Gamma \vdash t_1 \sqsubseteq t \quad \Gamma \vdash t \sqsubseteq t_2}{\Gamma \vdash (t_1, t_2)}$$

Temporal Logic Rules In the following rules v_t is a “fresh” variable:

$$\frac{\Gamma \vdash t \triangleleft t_1 \quad \Gamma \vdash \varphi^{t_1}}{\Gamma \vdash \bigcirc \varphi^t} \bigcirc R \quad \frac{\Gamma \vdash t \sqsubseteq t_1 \quad \Gamma \vdash \psi^{t_1} \quad \Gamma \vdash \varphi^{[t, t_1)}}{\Gamma \vdash \varphi \mathbf{U} \psi^t} \mathbf{UR}$$

$$\frac{\Gamma, t \triangleleft v_t, \varphi^{v_t}, \Delta \vdash \delta^a}{\Gamma, \bigcirc \varphi^t, \Delta \vdash \delta^a} \bigcirc L \quad \frac{\Gamma, t \sqsubseteq v_t, \psi^{v_t}, \varphi^{[t, v_t)}, \Delta \vdash \delta^a}{\Gamma, \varphi \mathbf{U} \psi^t, \Delta \vdash \delta^a} \mathbf{UL}$$

$$\frac{\Gamma \vdash t_1 \triangleleft t \quad \Gamma \vdash \varphi^{t_1}}{\Gamma \vdash \mathbf{Y} \varphi^t} \mathbf{YR} \quad \frac{\Gamma \vdash t_1 \sqsubseteq t \quad \Gamma \vdash \psi^{t_1} \quad \Gamma \vdash \varphi^{[t_1, t)}}{\Gamma \vdash \varphi \mathbf{S} \psi^t} \mathbf{SR}$$

$$\frac{\Gamma, v_t \triangleleft t, \varphi^{v_t}, \Delta \vdash \delta^a}{\Gamma, \mathbf{Y} \varphi^t, \Delta \vdash \delta^a} \mathbf{YL} \quad \frac{\Gamma, v_t \sqsubseteq t, \psi^{v_t}, \varphi^{(v_t, t]}, \Delta \vdash \delta^a}{\Gamma, \varphi \mathbf{S} \psi^t, \Delta \vdash \delta^a} \mathbf{SL}$$

Timed Rules The rules for the comparison and freeze operators are shown below. We only show some of the comparison rules, as they reflect the axioms in Fig. 2. Furthermore, TPTL-DIST’s $(\bigcirc)L$, $(\bigcirc)L$, $(\bigcirc)L$, $(\bigcirc)L$, $(\bigcirc)R$, $(\bigcirc)R$, $(\bigcirc)R$, and $(\bigcirc)R$ rules allow instantiating intervals, some of which are shown below (the others are of the same form).

$$\frac{}{\Gamma, t_1 \sqsubseteq t_2, \Delta \vdash t_1 \sqsubseteq t_2^a} \text{var} \sqsubseteq \frac{\Gamma \vdash \varphi[v_t \backslash t]^t}{\Gamma \vdash v_t \cdot \varphi^t} \cdot R$$

$$\frac{}{\Gamma \vdash t_1 \bullet t_2 \equiv t_2 \bullet t_1^a} \bullet \text{sym} \quad \frac{\Gamma, \varphi[v_t \backslash t]^t, \Delta \vdash \psi^a}{\Gamma, v_t \cdot \varphi^t, \Delta \vdash \psi^a} \cdot L$$

$$\frac{\Gamma, \varphi^t, \Delta \vdash \psi^a \quad \Gamma, \Delta \vdash t_1 \sqsubseteq t \quad \Gamma, \Delta \vdash t \sqsubseteq t_2}{\Gamma, \varphi^{[t_1, t_2]}, \Delta \vdash \psi^a} \llbracket L$$

$$\frac{\Gamma, t_1 \sqsubseteq v_t, v_t \sqsubseteq t_2 \vdash \phi^{v_t}}{\Gamma \vdash \phi^{[t_1, t_2]}} \llbracket R$$

Predicate Logic Rules The predicate rules are as follows, where v' is a “fresh” variable and X is a term of type T :

$$\frac{\Gamma \vdash \varphi[v \setminus v']^a}{\Gamma \vdash \forall v : T. \varphi^a} \quad \forall R \quad \frac{\Gamma, \varphi[v \setminus X]^{a1}, \Delta \vdash \psi^a}{\Gamma, \forall v : T. \varphi^{a1}, \Delta \vdash \psi^a} \quad \forall L$$

$$\frac{\Gamma \vdash \varphi[v \setminus X]^a}{\Gamma \vdash \exists v : T. \varphi^a} \quad \exists R \quad \frac{\Gamma, \varphi[v \setminus v']^t, \Delta \vdash \psi^a}{\Gamma, \exists v : T. \varphi^t, \Delta \vdash \psi^a} \quad \exists L$$

Induction The induction rule is as follows, where v_t does not occur free in Γ :

$$\frac{\Gamma, v_t \cdot \varphi^{[0, v_t)} \vdash \varphi^{v_t}}{\Gamma \vdash \varphi^{v_t}} \quad ind$$

Classical Rules

$$\frac{}{\Gamma \vdash \varphi \vee \neg \varphi^a} \quad lem$$

Derived Rules In all the rules below, v_t stands for a “fresh” variable.

$$\frac{\Gamma \vdash \varphi^{t1} \quad \Gamma \vdash t \sqsubseteq t_1}{\Gamma \vdash \Diamond \varphi^t} \quad \Diamond R \quad \frac{\Gamma, \varphi^{v_t}, t_1 \sqsubseteq v_t, \Delta \vdash \psi^t}{\Gamma, \Diamond \varphi^{t1}, \Delta \vdash \psi^t} \quad \Diamond L$$

$$\frac{\Gamma, t \sqsubseteq v_t \vdash \varphi^{v_t}}{\Gamma \vdash \Box \varphi^t} \quad \Box R \quad \frac{\Gamma, \varphi^{t2}, \Delta \vdash \psi^t \quad \Gamma, \Delta \vdash t_1 \sqsubseteq t_2}{\Gamma, \Box \varphi^{t1}, \Delta \vdash \psi^t} \quad \Box L$$

$$\frac{\Gamma \vdash \varphi^{t1} \quad \Gamma \vdash t \sqsubseteq t_1 \quad \Gamma \vdash t_1 \sqsubseteq t_0 \bullet t}{\Gamma \vdash \Diamond_{t_0} \varphi^t} \quad \Diamond_R$$

$$\frac{\Gamma, \varphi^{v_t}, t_1 \sqsubseteq v_t, v_t \sqsubseteq t_0 \bullet t_1, \Delta \vdash \psi^t}{\Gamma, \Diamond_{t_0} \varphi^{t1}, \Delta \vdash \psi^t} \quad \Diamond_L$$

$$\frac{\Gamma, t \sqsubseteq v_t, v_t \sqsubseteq t_0 \bullet t \vdash \varphi^t}{\Gamma \vdash \Box_{t_0} \varphi^t} \quad \Box_R$$

$$\frac{\Gamma, \varphi^{t2}, \Delta \vdash \psi^t \quad \Gamma, \Delta \vdash t_1 \sqsubseteq t_2 \quad \Gamma, \Delta \vdash t_2 \sqsubseteq t_0 \bullet t_1}{\Gamma, \Box_{t_0} \varphi^{t1}, \Delta \vdash \psi^t} \quad \Box_L$$

$$\frac{\Gamma \vdash \varphi^{t1} \quad \Gamma \vdash t_1 \sqsubseteq t}{\Gamma \vdash \blacklozenge \varphi^t} \quad \blacklozenge R \quad \frac{\Gamma, \varphi^{v_t}, v_t \sqsubseteq t_1, \Delta \vdash \psi^t}{\Gamma, \blacklozenge \varphi^{t1}, \Delta \vdash \psi^t} \quad \blacklozenge L$$

$$\frac{\Gamma, v_t \sqsubseteq t \vdash \varphi^{v_t}}{\Gamma \vdash \blacksquare \varphi^t} \quad \blacksquare R \quad \frac{\Gamma, \varphi^{t2}, \Delta \vdash \psi^t \quad \Gamma, \Delta \vdash t_2 \sqsubseteq t_1}{\Gamma, \blacksquare \varphi^{t1}, \Delta \vdash \psi^t} \quad \blacksquare L$$

$$\frac{\Gamma \vdash \varphi^{t1} \quad \Gamma \vdash t_1 \sqsubseteq t}{\Gamma \vdash \blacktriangleleft \varphi^t} \quad \blacktriangleleft R \quad \frac{\Gamma, \varphi^{v_t}, v_t \sqsubseteq t_1, \Delta \vdash \psi^t}{\Gamma, \blacktriangleleft \varphi^{t1}, \Delta \vdash \psi^t} \quad \blacktriangleleft L$$

C Proof of eq. 3

This section presents a fully worked out proof of eq. 3, a sketch of which was presented in Sec. 3.6.

Let

$$\begin{aligned}\Gamma_0 &\equiv (Eq. 1)^t, (Eq. 2)^t \\ \Gamma_1 &\equiv \Gamma_0, t \sqsubseteq v_t, \mathbf{send}(a, d, [b])^{v_t}, \mathbf{recv}(b, d, a)^{v'_t}, v_t \sqsubseteq v'_t, v'_t \sqsubseteq T \bullet v_t \\ \Gamma_2 &\equiv \Gamma_1, \mathbf{send}(b, d, [c])^{v'_t} \\ \Gamma_3 &\equiv \Gamma_2, \mathbf{recv}(c, d, b)^{v''_t}, v'_t \sqsubseteq v''_t, v''_t \sqsubseteq T \bullet v'_t\end{aligned}$$

To prove $\Gamma_0 \vdash Eq. 3^t$, we start with a few introductions to in particular get the assumption that $\mathbf{send}(a, d, [b])^{v_t}$:

$$\frac{\frac{\frac{\Gamma_0, t \sqsubseteq v_t, \mathbf{send}(a, d, [b])^{v_t} \vdash \Diamond_{2T} \mathbf{recv}(c, d, b)^{v_t}}{\Gamma_0, t \sqsubseteq v_t \vdash \mathbf{send}(a, d, [b]) \rightarrow \Diamond_{2T} \mathbf{recv}(c, d, b)^{v_t}} \rightarrow R}{\Gamma_0 \vdash \Box(\mathbf{send}(a, d, [b]) \rightarrow \Diamond_{2T} \mathbf{recv}(c, d, b))^t} \Box R}{\Gamma_0 \vdash Eq. 3^t} \forall R(\times 4)$$

We now instantiate Eq. 1 to, in particular, get the assumption $\mathbf{recv}(b, d, a)^{v'_t}$:

$$\frac{\frac{\frac{\Gamma_1 \vdash \Diamond_{2T} \mathbf{recv}(c, d, b)^{v_t}}{\Gamma_0, t \sqsubseteq v_t, \mathbf{send}(a, d, [b])^{v_t}, \phi_3^{v'_t}, v_t \sqsubseteq v'_t, v'_t \sqsubseteq T \bullet v_t \vdash \Diamond_{2T} \mathbf{recv}(c, d, b)^{v_t}} \rightarrow L \ \& \ \in R}{\Gamma_0, t \sqsubseteq v_t, \mathbf{send}(a, d, [b])^{v_t}, \phi_2^{v_t} \vdash \Diamond_{2T} \mathbf{recv}(c, d, b)^{v_t}} \Diamond_L}{\frac{\frac{\Gamma_0, t \sqsubseteq v_t, \mathbf{send}(a, d, [b])^{v_t}, \phi_1^{v_t} \vdash \Diamond_{2T} \mathbf{recv}(c, d, b)^{v_t}}{\Gamma_0, t \sqsubseteq v_t, \mathbf{send}(a, d, [b])^{v_t} \vdash \Diamond_{2T} \mathbf{recv}(c, d, b)^{v_t}} \rightarrow L \ \& \ var}{\Gamma_0, t \sqsubseteq v_t, \mathbf{send}(a, d, [b])^{v_t} \vdash \Diamond_{2T} \mathbf{recv}(c, d, b)^{v_t}} \forall L(\times 4) \ \& \ \Box L \ \& \ var \sqsubseteq}$$

where

$$\begin{aligned}\phi_1 &\equiv \mathbf{send}(a, d, [b]) \rightarrow \phi_2 \\ \phi_2 &\equiv \Diamond_T(\phi_3) \\ \phi_3 &\equiv b \in [b] \rightarrow \mathbf{recv}(b, d, a)\end{aligned}$$

We instantiate Eq. 2 to, in particular, get the assumption $\mathbf{send}(b, d, [c])^{v'_t}$:

$$\frac{\frac{\frac{\Gamma_2 \vdash \Diamond_{2T} \mathbf{recv}(c, d, b)^{v_t}}{\Gamma_1, \phi_4^{v'_t} \vdash \Diamond_{2T} \mathbf{recv}(c, d, b)^{v_t}} \rightarrow L \ \& \ var}{\Gamma_1 \vdash \Diamond_{2T} \mathbf{recv}(c, d, b)^{v_t}} \sqsubseteq trans \ \& \ var \sqsubseteq}{\Gamma_1 \vdash t \sqsubseteq v_t^{v'_t}} \forall L(\times 4) \ \& \ \Box L$$

where

$$\phi_4 \equiv \mathbf{recv}(b, d, a) \rightarrow \mathbf{send}(b, d, [c])$$

We instantiate Eq. 1 once more, to in particular get the assumption $\mathbf{recv}(c, d, b)^{v''_t}$:

$$\frac{\frac{\frac{\Gamma_3 \vdash \Diamond_{2T} \mathbf{recv}(c, d, b)^{v_t}}{\Gamma_2, \phi_7^{v''_t}, v'_t \sqsubseteq v''_t, v''_t \sqsubseteq T \bullet v'_t \vdash \Diamond_{2T} \mathbf{recv}(c, d, b)^{v_t}} \rightarrow L \ \& \ \in R}{\Gamma_2, \phi_6^{v'_t} \vdash \Diamond_{2T} \mathbf{recv}(c, d, b)^{v_t}} \Diamond_L}{\frac{\frac{\Gamma_2, \phi_5^{v'_t} \vdash \Diamond_{2T} \mathbf{recv}(c, d, b)^{v_t}}{\Gamma_2, \phi_5^{v'_t} \vdash \Diamond_{2T} \mathbf{recv}(c, d, b)^{v_t}} \rightarrow L \ \& \ var}{\Gamma_2 \vdash \Diamond_{2T} \mathbf{recv}(c, d, b)^{v_t}} \sqsubseteq trans \ \& \ var \sqsubseteq}{\Gamma_1 \vdash t \sqsubseteq v_t^{v''_t}} \forall L(\times 4) \ \& \ \Box L$$

where

$$\begin{aligned}\phi_5 &\equiv \mathbf{send}(b, d, [c]) \rightarrow \phi_6 \\ \phi_6 &\equiv \Diamond_T(\phi_7) \\ \phi_7 &\equiv c \in [c] \rightarrow \mathbf{recv}(c, d, b)\end{aligned}$$

We finally conclude using in particular the derived rule $\Diamond_-R$, where $\Gamma_3 \vdash v_t'' \sqsubseteq 2T \bullet v_t^{vt}$ is trivial but tedious to prove using among other rules, $\sqsubseteq trans$, $\bullet assoc$, and $var \sqsubseteq$:

$$\frac{\frac{\Gamma_3 \vdash v_t \sqsubseteq v_t''^{vt} \quad \sqsubseteq trans \ \& \ var \sqsubseteq}{\Gamma_3 \vdash v_t'' \sqsubseteq 2T \bullet v_t^{vt}}}{\Gamma_3 \vdash \Diamond_{2T} \mathbf{recv}(c, d, b)^{vt}} \Diamond_-R$$